

复杂代码组

北京理工大学

我们本章来学习比较复杂的代码组。不同于其它大多数语言，Python 中的代码组不能使用大括号 {} 来控制类、函数或者其他逻辑判断，其最具特色的就是用缩进来表示代码组。我们可以使用 tab 或 space(空格键) 来表示缩进。缩进的空白数量是可变的，但是同一代码块的语句必须包含相同的缩进空白数量。

```
if True:
    print("True")
else:
    print("False")
```

在 Python 中，以下代码在执行时可能会报错：

```
if True:
    print("True")
else:
    print("Answer")
    print("False")
```

IndentationError: unexpected indent

这个错误提示可能是 tab 或者空格没对齐的问题。

```
if True:
    print("True")
else:
    print("Answer")
    print("False")
```

IndentationError: unindent does not match any outer indentation level

在输入上面这段程序时，如果第四行使用制表符 tab 缩进，第五行使用空格键 space 缩进，尽管表面上看起来是对齐了，但在执行程序时，还是会报错。上面程序报错提示使用的缩进方式不一致，有的是制表符 tab 缩进，有的是空格键 space 缩进，改为一致即可。尽管部分编译器会自动把制表符 tab 和空格键 space 统一起来，但考虑到程序可能会在不同的编译器中运行，即使不为语法问题，为了提高程序语言的一致性，我们还是建议在每个缩进层次使用单个 tab 或两个空格或四个空格，不要混用。另外，切记在 Python 的代码块中必须使用相同数目的行首缩进空格数。

缩进相同的一组语句构成一个代码块，我们称之为代码组。像 if、while、def 和 class 这样的复合语句，首行以关键字开始，以冒号结束，该行之后的一行或多行代码构成代码组。首行及后面的代码组组成一个程序模块。在本章，我们将学习判断语句、循环语句及函数调用等比较复杂的代码组。

我们先介绍一些准备知识。先来看看标识符。在 Python 里，标识符由字母、数字以及下划线组成，区分大小写字母，但不能以数字开头，标识符不能使用空格，不能使用保留字符。以下划线开头的标识符是有特殊意义的：以单下划线 _ 开头代表不能直接访问的类属性，需要通过类提供的接口进行访问，不能通过 `from xxx import xxx` 导入；以双下划线开头代表类的私有成员；以双下划线开头和结尾代表特殊方法的专用标识，如 `__init__()` 代表类的构造函数。

使用分号，Python 可以在同一行显示多条语句。比如：

```
x = 'Why '; y = 'do you '; z = 'learn Python?';  
print(x + y + z)
```

Why do you learn Python?

Python 语句中一般以换行作为语句的结束符，但是我们可以使用斜杠 \ 将一行的语句分为多行显示。比如说，

```
vegetables = 'potato ' + \
             'tomato ' + \
             'cabbage'
vegetables
```

'potato tomato cabbage'

语句中包含 [], {} 或 () 括号就不需要使用多行连接符。

```
WeekDays = ['Monday', 'Tuesday', 'Wednesday',
            'Thursday', 'Friday']
WeekDays
```

['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday']

我们之前提到过，Python 使用单引号 (')、双引号 (")、三引号 (''' 或 ''') 来表示字符串，但引号的开始与结束必须是相同类型。其中，三引号可以由多行组成，常用于文档字符串，在代码组中一般被当做注释。三引号之间的内容可以直接换行，而单引号和双引号之间换行需要使用斜杠符号。

```
word = 'word'
sentence = "这是一个句子。"
paragraph = """这是一个段落。
包含了多个语句"""
```


Python 中，多行注释一般使用三个单引号 (' ' ') 或三个双引号 (" " ")。

```
'''
```

这是多行注释，使用单引号。

这是多行注释，使用单引号。

这是多行注释，使用单引号。

```
'''
```

```
"""
```

这是多行注释，使用双引号。

这是多行注释，使用双引号。

这是多行注释，使用双引号。

```
"""
```

对于单行注释，我们可以简单地采用 # 开头即可。

```
# 第一个注释  
print("Hello, Dr. Lee!")
```

Hello, Dr. Lee!

注释也可以在语句或表达式行末。

```
name = "Dr. Lee" # 这位是李博士。"
```

函数之间或类的方法之间可以使用空行分隔，表示一段新的代码的开始。类和函数入口之间也用一行空行分隔，以突出函数入口的开始。与代码缩进不同，空行并不是 Python 语法的一部分。如果书写时不插入空行，Python 解释器运行也不会出错。但是空行的作用在于分隔两段不同功能或含义的代码，便于日后代码的维护或重构。空行也是程序代码的一部分。

除了输出函数 `print`，python 中还有输入函数 `input`。它接受一个标准输入数据，返回为 `string` 类型。下面的程序执行后，首先会显示双引号内的汉字，`\n` 实现换行，用户在方框内进行输入。最后按回车键退出输入，并将显示输入内容。

```
input("按下回车键退出，其他任意键显示输入的内容\n")
```

`input` 函数的调用格式为 `input(["提示信息"])`。比如说：输入 = `input("请您输入一个数字:")`。

条件语句

条件语句通过判断指定条件是否为真 (True 或 False) 来决定是否继续执行后续指令。任何非 0 或者非空的值为 True; 0 或者空值为 False。在条件语句中, if 语句基本形式为:

if 判断条件:

 执行语句

elif 判断条件:

 执行语句

else:

 执行语句

其中，“判断条件”为真时，执行后续语句，以相同的缩进表示属于同一后续模块；“判断条件”不成立时，执行 else 的后续模版。注意，else 为可选语句。当判断条件为多个值时，可以使用以下形式：

```
if 判断条件:
```

```
    执行语句
```

```
elif 判断条件1:
```

```
    执行语句1
```

```
elif 判断条件2:
```

```
    执行语句2
```

```
elif 判断条件3:
```

```
    执行语句3
```

```
else:
```

```
    执行语句
```

我们先来看几个简单的例子。

例 5.1.1:

```
gender = '男'  
# gender = '女'  
# 表示注释  
if gender == '男':  
    sex = 1  
else:  
    sex = 0  
print(sex)
```

1

例 5.1.2:

```
样貌 = '不帅'
人品 = '不好'
if 样貌 == '不帅' and 人品 == '不好':
    print('我觉得不大行')
else:
    print('可以考虑考虑')
```

我觉得不大行

```
样貌 = '帅呆了'
人品 = '不好'
if 样貌 == '帅呆了':
    print('可以考虑考虑')
else:
    if 人品 == '好':
        print('可以考虑考虑')
    else:
        print('还是算了吧')
```

可以考虑考虑

例 5.1.3: 中国人民大学 2018 年在北京地区文科录取分数线为 665 分, 理科分数线是 674 分; 在安徽地区文科录取分数线是 651 分, 理科分数线是 668 分。

```
生源 = ['北京','安徽']
科目 = ['文科','理科']
分数 = [665,674,651,668]
学生 = ['北京','文科',668]

if 学生[0] not in 生源:
    print('该考生不是来自北京或安徽')
else:
    print('该考生来自北京或者安徽')
```

该考生来自北京或者安徽

```
学生 = ['北京','工科',668]

if 学生[1] not in 科目:
    print('信息输入错误')
else:
    print('信息输入正确')
```

信息输入错误

注意, if 语句的判断条件可以用 > (大于)、<(小于)、== (等于)、>= (大于等于)、<= (小于等于) 来表示其关系。

```
学生 = ['北京','工科',699]
if 学生[2] >= max(分数):
    print('该生如果来自北京或者安徽的话, 肯定可以考上人民大学')
```

该生如果来自北京或者安徽的话, 肯定可以考上人民大学

从分数线看来, 理科的分数线比文科略高, 北京地区比安徽地区略高。

例 5.1.4：法定结婚年龄是指根据《中华人民共和国婚姻法》第六条规定的男方结婚年龄不得早于 22 周岁，女不得早于 20 周岁。

```
性别 = '男'; 年龄 = 25 # 仅需使用第一行或第二行
性别 = '女'; 年龄 = 15

if (性别 == '男' and 年龄 >= 22) or (性别 == '女' and 年龄 >= 20):
    print('祝贺你，你可以登记结婚了')
elif (性别 == '男'):
    print('不要这么急嘛，小伙子，你还小，至少要再等 %d 年。'%(22-年龄))
else:
    print('不要这么急，小姑娘，你还小，至少要再等 %d 年。'%(20-年龄))
```

不要这么急，小姑娘，你还小，至少要再等 5 年。

例 5.1.5: 《民法通则》规定: 不满 10 周岁, 为无民事行为能力人; 10 周岁至 18 或 16 周岁, 为限制民事行为能力人; 年满 18 周岁或者对于年满 16 周岁而不满 18 周岁, 但是以自己的劳动收入为主要生活来源的公民, 为完全民事行为能力人, 可以独立进行民事活动。

```
年龄 = 17
```

```
if 年龄 < 10:
```

```
    print('无民事行为能力')
```

```
elif 年龄 >= 18:
```

```
    print("完全民事行为能力")
```

```
elif 年龄 <= 16:
```

```
    print("限制民事行为能力")
```

```
else:
```

```
    以自己的劳动收入作为主要生活来源 = \
```

```
    input("是否以自己的劳动收入作为主要生活来源, 请输入'是' 或者'否': ")
```

```
    if 以自己的劳动收入作为主要生活来源 == '是':
```

```
        print("完全民事行为能力")
```

```
    else:
```

```
        print("限制民事行为能力")
```

是否以自己的劳动收入作为主要生活来源, 请输入'是' 或者'否': 否

例 5.1.6: 北京市公安局公安交通管理局对部分机动车采取尾号限行的交通管理措施。其中, 2018 年 10 月 8 日至 2019 年 1 月 6 日的星期一至星期五, 限行机动车车牌尾号分别为: 2 和 7、3 和 8、4 和 9、5 和 0、1 和 6。

```
车牌号 = '京A-1234'  
尾号 = int(车牌号[-1]) # 取尾号并将其转换为整数形式  
  
if 尾号 in [2,7]:  
    print('周一限行')  
elif 尾号 in [3,8]:  
    print('周二限行')  
elif 尾号 in [4,9]:  
    print('周三限行')  
elif 尾号 in [5,0]:  
    print('周四限行')  
else:  
    print('周五限行')
```

周三限行

例 5.1.7:

```
季节 = '冬天'
天气 = '晴天'
温度 = 0

if 季节 in '冬天' and 温度 >= 10:
    print('今天很暖和。')
elif 天气 in '晴天':
    print('虽然气温很低，但是太阳晒在身上暖洋洋的。')
else:
    print('今天不宜出门。')
```

虽然气温很低，但是太阳晒在身上暖洋洋的。

例 5.1.8: Python 复合布尔表达式计算采用短路规则, 即如果通过前面的部分已经计算出整个表达式的值, 则后面的部分不再计算。下面的代码将正常执行, 不会报除数为零的错误:

```
a=0
b=1
if ( a > 0 ) and ( b / a > 2 ):
    print("a > , 0and b / a > 2")
else :
    print("a is less than or equal to 0")
```

a is less than or equal to 0

而下面的代码就会报错:

```
a=0
b=1
if ( a > 0 ) or ( b / a > 2 ):
    print("yes")
else:
    print("no")
```

循环语句

循环语句允许我们执行一个语句或语句组多次。Python 提供了两种循环：while 循环和 for 循环。当然，基于这两种循环可以构造嵌套循环。

在 Python 编程中，while 语句用于判断是否循环执行程序。当判断条件为真时，循环处理执行语句。执行语句可以是单个语句或语句块。判断条件可以是任何表达式，任何非零、或非空的值均为 True。当判断条件为 False 时，循环结束。其基本形式为：

```
while 判断条件:  
    执行语句
```

这里，while 语句和之前的 if 语句，以及即将要介绍的 for 语句是一样的，都无需使用 end 结束函数，我们使用不同的缩进来表示语句是否属于同一程序模块。我们来看一些例子。

while 语句

例 5.2.1:

```
number = 0
while (number < 3):
    print('The number is', number)
    number += 2
print('The number ends at', number)
```

The number is 0

The number is 2

The number ends at 4

例 5.2.2:

```
和 = 0
n = 0
while n <= 3:
    和 += n
    print(n, 和)
    n += 1
print(和)
```

0 0

1 1

2 3

3 6

6

例 5.2.3:

```
活着 = True
年龄 = int(0)
while 活着 and (年龄 <= 1E4):
    print('我一直等你')
    年龄 += 5E3
```

我一直等你

我一直等你

我一直等你

在上面的程序中，如果没有 (年龄 <= 1E4) 这个限制，程序会一直不停地执行下去。此时，就不得不使用 Ctrl+C 或 Interrupt 来结束程序了。

例 5.2.4:

```
列表 = [12, 37, 45, 64, 97, 104]
偶数 = []
奇数 = []
while len(列表) > 0:
    number = 列表.pop()
    if (number % 2 == 0):
        偶数.append(number)
    else:
        奇数.append(number)
print('偶数为 %s ' %偶数)
print('奇数为 %s ' %奇数)
```

偶数为 [104, 64, 12]

奇数为 [97, 45, 37]

例 5.2.5:

```
var = 0
number = []
while var == 1:
    num = input("请输入数字: ")
    number.append(int(num))
    print("您输入的数字为", num, ", 现在序列被更新为", number)
```

使用 while 语句时还可以使用两个重要的命令：continue 以及 break。其中，continue 用于跳过当前循环，开始下一次循环，break 则是用于退出本轮循环。此外，“判断条件”还可以是个常值，若其非零，表示循环必定成立。下面是具体的例子。

例 5.2.6:

```
var = 0
number = []
while var == 0:
    num = input("请输入数字:")
    number.append(num)
    print("您输入的数字为", num, ", 现在序列被更新为", number)
    if len(number) >= 2:
        break
```

请输入数字:2

您输入的数字为 2，现在序列被更新为 ['2']

请输入数字:2

您输入的数字为 2，现在序列被更新为 ['2', '2']

例 5.2.7:

```
n = 0
while n < 10:
    n += 1
    if n % 2 == 0:
        continue
    print(n, end=" ")
```

1 3 5 7 9

```
活着 = True
年龄 = 0
while 活着 & (年龄 <= 1E4):
    年龄 += 1
    if 年龄 % 5E3 != 0:
        continue
    print('花儿都谢了')
```

花儿都谢了

花儿都谢了

例 5.2.8:

```
活着 = True
```

```
年龄 = 0
```

```
while 活着:
```

```
    年龄 += 5E3
```

```
    if 年龄 > 1E4:
```

```
        break
```

```
    print('等你一万年')
```

等你一万年

等你一万年

```
活着 = True
```

```
年龄 = 0
```

```
while 活着:
```

```
    年龄 += 5E3
```

```
    if 年龄 > 1E4:
```

```
        break
```

```
    print('等你一万年')
```

等你一万年

在上面的例子中，我们改变了 `print` 函数的位置。大家可以仔细对比一下差别。

通过上面的一些例子可以看到，`break` 语句可以在循环过程中直接退出循环，而 `continue` 语句可以提前结束本轮循环，并直接开始下一轮循环。这两个语句通常都必须配合 `if` 语句使用。但要注意不要滥用 `break` 和 `continue` 语句。`break` 和 `continue` 会造成代码执行逻辑分叉过多，容易出错。大多数循环并不需要用到 `break` 和 `continue` 语句，上面的两个例子都可以通过改写循环条件或者修改循环逻辑，去掉 `break` 和 `continue` 语句。如果代码写得有问题，会让程序陷入“死循环”，也就是永远循环下去。这时可以用 `Ctrl+C` 或者通过 Kernel 中的 `Interrupt` 命令退出程序，强制结束 Python 进程。

另外，`while...else` 在循环条件为 `false` 时执行 `else` 语句块。格式如下：其基本形式为：

```
while 判断条件:
```

```
    执行语句
```

```
else:
```

```
    执行语句
```

例 5.2.9:

```
Year = 0
MaxYear = 1E4
while Year <= MaxYear:
    Year += 1
    if (Year % 5E3 == 0):
        print('我已经等待%5d年了。'%Year)
    else:
        print('我已经等待%5d年了。'%Year)
```

我已经等待 5000 年了。

我已经等待 10000 年了。

我已经等待 10001 年了。

例 5.2.10 (猜大小):

```
import random
s = int(random.randint(1, 5))
m = int(input('请您输入一个整数:'))
while m != s:
    if m > s:
        print('您输入的数字太大了')
        m = int(input('输入一个较小的整数:'))
    if m < s:
        print('您输入的数字太小了')
        m = int(input('输入一个较大的整数:'))
print('恭喜您, 猜对了。')
```

请您输入一个整数:3

您输入的数字太大了

输入一个较小的整数:2

您输入的数字太大了

输入一个较小的整数:1

恭喜您, 猜对了。

例 5.2.11 (猜拳小游戏):

```
import random
random.seed(200)
while True:
    随机数 = int(random.randint(1, 3))
    if 随机数 == 1:
        电脑 = "石头"
    elif 随机数 == 2:
        电脑 = "剪刀"
    else:
        电脑 = "布"
    我 = input("输入 '石头'、'剪刀'、'布'，或者输入 '结束' 结束游戏:")
    选择 = ['石头', "剪刀", "布"]
    if (我 not in 选择) and (我 != '结束'):
        print("输入错误，请重新输入！")
```

接上页

```
elif 我( not in 选择) and (我 == '结束'):  
    print("\n游戏结束了")  
    break  
elif 我 == 电脑:  
    print("电脑出了:  " + 电脑 + ", 平局! ")  
elif (我 == '石头' and 电脑 == '剪刀') or (我 == '剪刀' and 电脑 == '布') or (我 == '布' and 电脑 == '石头'):  
    print("电脑出了:  " + 电脑 + ", 你赢了! ")  
else:  
    print("电脑出了:  " + 电脑 + ", 你输了! ")
```

输入'石头'、'剪刀'、'布', 或者输入'结束' 结束游戏: 石头

电脑出了: 石头, 平局!

输入'石头'、'剪刀'、'布', 或者输入'结束' 结束游戏: 剪刀

电脑出了: 石头, 你输了!

输入'石头'、'剪刀'、'布', 或者输入'结束' 结束游戏: 结束

游戏结束了

for 语句

在循环语句中，for 循环可以遍历任何序列的项目，如一个列表、元组或一个字符串。使用 for...in 循环，可以把列表、元组或者字符串中的所有元素依次迭代出来。其循环的语法格式如下：

```
for iterating_var in sequence:  
    statement(s)
```

for 循环的其它用法基本和 while 语句是一致的。

例 5.2.12:

```
sum = 0  
for x in range(5):  
    sum += x  
print(sum)
```

这里用到了 range 函数。可以通过 list 函数来查看 range 函数的结果。

```
list(range(5))
```

```
[0, 1, 2, 3, 4]
```

```
names = ['数据', '科学']  
for name in names:  
    print(name, end = ' ')
```

数据科学

```
str1 = ['习近平', '新时期', '中国特色', '社会主义', '思想']  
for name in str1:  
    print(name, end = ' ')
```

习近平新时代中国特色社会主义思想

```
for letter in 'STAT':  
    print('letter:', letter)
```

letter: S

letter: T

letter: A

letter: T

```
一线城市 = ['北京', '上海', '广州', '深圳']  
for 城市 in 一线城市:  
    print('一线城市:', 城市)  
print("程序结束!")
```

一线城市: 北京

一线城市: 上海

一线城市: 广州

一线城市: 深圳

程序结束!

另外一种执行循环的遍历方式是通过下标索引。

例 5.2.13:

```
一线城市 = ['北京', '上海', '广州', '深圳']  
for 城市 in range(len(一线城市)):  
    print('一线城市:', 一线城市[城市])  
print("程序结束!")
```

一线城市: 北京

一线城市: 上海

一线城市: 广州

一线城市: 深圳

程序结束!

```
一线城市[0][1]
```

'京'

与 while...else 一样，在 python 中可以使用 for...else 语句，用法基本一样：在 for 语句正常执行完毕（不是通过 break 跳出而中断的）的情况下执行 else 中的语句。

例 5.2.14:

```
一线城市 = ['北京', '上海', '广州', '深圳']  
print('一线城市: ', end = ' ')  
for 城市 in range(len(一线城市)):  
    print( 一线城市[城市], end = ' ' )  
else:  
    print("\n其他为二线或二线以下城市")
```

一线城市: 北京上海广州深圳

其他为二线或二线以下城市

例 5.2.15 居民身份证是国家法定的证明公民个人身份的证件。为了堵塞和制止假居民身份证的流通和使用,《中华人民共和国国家标准 (GB11643-1999)》中有关公民身份号码的规定表示,公民身份号码是特征组合码,由十七位数字本体码和一位数字校验码组成。

在校验身份证号码时,可以将身份证号码前 17 位数分别乘以 7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2, 然后讲这些乘积的结果相加,用加出来和除以 11。余数可能是 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 对应最后的校验码分别是 1, 0, X, 9, 8, 7, 6, 5, 4, 3, 2。

我们提供两个身份证号码：53010219200508011x、11012020180521433X。请检查这两个身份证号码的校验码是否正确。

```
身份证编号 = '53010219200508011x'
# 身份证编号 = '11012020180521433X'
乘数 = [7,9,10,5,8,4,2,1,6,3,7,9,10,5,8,4,2]
校验码 = [1,0,'x',9,8,7,6,5,4,3,2]
和 = 0;
for i in range(17):
    和 += int(身份证编号[i])*乘数[i]
if str(校验码[(和%11)].upper())==str(身份证编号[-1]).upper():
    print('校验码正确')
else:
    print('校验码错误，正确的校验码应该为%s。' %str(校验码[(和%11]))
```

校验码正确

循环嵌套

我们学习了 while 和 for 循环语句之后，就可以使用循环嵌套语句了。Python 语言允许在一个循环体里面嵌入另一个循环。在 for 和 while 循环嵌套语句中，语法基本一致。

Python for 循环嵌套语法：

```
for iterating_var in sequence:
    for iterating_var in sequence:
        statement(s)
        statement(s)
```

Python while 循环嵌套语法：

```
while expression:
    while expression:
        statement(s)
    statement(s)
```

例 5.2.16: 打印金字塔结构。

```
MaxNum = 3  
for i in range(2,MaxNum+2):  
    for k in range(1,i):  
        print(k, end=" ")  
    print("\n")
```

1

1 2

1 2 3

思考一下，在上面程序的第二行中，为什么需要加上 2？

```
i=1
MaxNum = 5
while 1:
    if MaxNum % 2 == 0:
        print('MaxNum必须是一个奇数。')
        MaxNum += 1
    break
# 如果MaxNum是偶数的话，自动增加一层。
Median = MaxNum//2 + 1
while i<=MaxNum:
    if i<= Median:
        print ("0"*i)
    elif i<= MaxNum:
        j=i-2*(i-Median)
        print("0"*j)
    i+=1
```

输出

O

OO

OOO

OO

O

例 5.2.17：两种不同的冒泡排序算法（本质相同）。

第一种

```
arrays = list(range(4,0,-1))
for i in range(len(arrays)):
    for j in range(i+1):
        if arrays[i] < arrays[j]:
            arrays[i], arrays[j] = arrays[j], arrays[i]
    print(i, j, arrays)
```

0 0 [4, 3, 2, 1]

1 0 [3, 4, 2, 1]

1 1 [3, 4, 2, 1]

2 0 [2, 4, 3, 1]

2 1 [2, 3, 4, 1]

2 2 [2, 3, 4, 1]

3 0 [1, 3, 4, 2]

3 1 [1, 2, 4, 3]

第二种

```
array = list(range(4,0,-1))
L = len(array)
for i in range(L):
    for j in range(L-i):
        if array[L-j-1]<array[L-j-2]:
            array[L-j-1],array[L-j-2]=array[L-j-2],array[L-j-1]
        print(i,j,array)
```

0 0 [4, 3, 1, 2]

0 1 [4, 1, 3, 2]

0 2 [1, 4, 3, 2]

0 3 [2, 4, 3, 1]

1 0 [2, 4, 1, 3]

1 1 [2, 1, 4, 3]

1 2 [1, 2, 4, 3]

2 0 [1, 2, 3, 4]

例 5.2.18：判断素数。

```
起点 = 10;
终点 = 20;
for num in range(起点, 终点):
    for i in range(2,num):
        if num%i == 0:
            #j=num/i
            #print('%d equals %d * %d' % (num,i,j))
            break
    else:
        print(num, '是素数')
```

11 是素数

13 是素数

17 是素数

19 是素数

例 5.2.18

```
起点 = 10;
终点 = 20;
素数 = 起点
while(素数 < 终点):
    j = 2
    while(j <= (素数/j)):
        if not(素数%j):
            break
    j += 1
    if (j > 素数/j) :
        print(素数, "是素数")
    素数 += 1
```

11 是素数

13 是素数

17 是素数

19 是素数

例 5.2.18

```
起点 = 10;
终点 = 20;
素数 = [];
for i in range(起点, 终点):
    for j in range(2,i):
        if (i%j==0):
            break
    else:
        素数.append(i)
print('%d到%d之间的素数有%s'%(起点, 终点, 素数))
```

10 到 20 之间的素数有 [11, 13, 17, 19]

例 5.2.18

```
起点 = 10
终点 = 20
素数 = []

for num in range(起点, 终点+1):
    snum = int(num*0.5+1)
    for i in range(2,snum):
        if num%i == 0:
            break
    else:
        素数.append(num)

print(起点, '到', 终点, '的质数有', 素数)
print(起点, '到', 终点, '有', len(素数), '个质数')
```

10 到 20 的质数有 [11, 13, 17, 19]

10 到 20 有 4 个质数

函数是组织好的、可重复使用的、用来实现单一或相关联功能的程序模块。函数能提高程序的模块性和代码的重复利用率。在 Python 中有三类函数：内置函数、用户自定义的一般函数以及用户自定义的匿名函数。

Python 内置了很多有用的函数，比如 print()，我们可以直接调用。

Table: Python 中部分内置函数

abs()	divmod()	input()	open()	staticmethod()
all()	enumerate()	int()	ord()	str()
any()	eval()	isinstance()	pow()	sum()
basestring()	execfile()	issubclass()	print()	super()
bin()	file()	iter()	property()	tuple()
bool()	filter()	len()	range()	type()
bytearray()	float()	list()	raw input()	unichr()
callable()	format()	locals()	reduce()	unicode()
chr()	frozenset()	long()	reload()	vars()
classmethod()	getattr()	map()	repr()	xrange()
cmp()	globals()	max()	reverse()	zip()
compile()	hasattr()	memoryview()	round()	_import_()
complex()	hash()	min()	set()	
delattr()	help()	next()	setattr()	

例 5.3.1: 函数调用。

```
abs(-20)
```

20

```
int('123')
```

123

```
float('12.34')
```

12.34

```
str(100)
```

'100'

```
bool(1)
```

True

```
bool('')
```

False

函数名其实就是指向一个函数对象的引用，完全可以把函数名赋给一个变量，相当于给这个函数起了一个“别名”：

```
a = abs  
a(-1)
```

1

调用函数的时候，如果传入的参数数量不对，会报 `TypeError` 的错误，并且 Python 会明确地告诉用户：`abs()` 能且仅能接受 1 个参数，但用户给出了两个 (`abs()` takes exactly one argument (2 given))；如果传入的参数数量是对的，但参数类型不能被函数所接受，也会报 `TypeError` 的错误，并且给出错误信息，比如输入 `abs('a')` 时，会显示 `bad operand type for abs(): 'str'`。可以在交互式命令行通过 `help(abs)` 查看 `abs` 函数的帮助信息。

用户也可以自定义函数。用户自定义函数就是用户自行创建的函数，它必须满足下面的简单规则：

- ❶ 函数代码块以 `def` 关键词开头，后接函数标识符名称和圆括号 `()`。
- ❷ 参数放在圆括号中间。
- ❸ 函数的第一行语句可以选择性地写一些函数说明。
- ❹ 函数内容以冒号起始，然后换行缩进。
- ❺ `return` 表达式结束函数，选择性地返回一个值给调用方。不带表达式的 `return` 相当于返回 `None`。

语法规则为：

```
def functionname(parameters):  
    "函数_文档字符串"  
    function_suite  
    return [expression]
```

例 5.3.2：去除字符串首尾的空格。

```
def trim(s):  
    while s[0] == ' ':  
        s = s[1:]  
    while s[-1] == ' ':  
        s = s[:-1]  
    return s
```

例 5.3.3：字符串拼接。

```
def plus(str1, str2):  
    total = str1 + str2  
    return total
```

定义函数时只给了函数一个名称，指定了函数里包含的参数和代码块结构。这个函数的基本结构完成以后，可以通过另一个函数调用执行。

例 5.3.4：调用自定义函数。

```
str = '   Susan   '
trim(str)
```

'Susan'

```
str1 = '数据'  
str2 = '科学'  
plus(str1, str2)
```

'数据科学'

```
plus(str2, str1)
```

'科学数据'

上面的测试结果暗示着变量的顺序非常重要，但是，如果使用如下的函数调用方式，则变量的顺序就不那么重要了。

```
plus(str1 = '数据', str2 = '科学')
```

'数据科学'

```
plus(str2 = '科学', str1 = '数据')
```

'数据科学'

函数中可以含有不同类型的参数。前面我们已经遇到了两种参数：必备参数和关键词参数。必备参数是指必须以正确的顺序传入函数。并且调用时的数量必须和声明时的一样的参数。比如说，在调用 `trim(str)` 函数，必须传入一个参数，不然会出现语法错误。关键字参数和函数调用关系紧密，函数调用使用关键字参数来确定传入的参数值。使用关键字参数允许函数调用时参数的顺序与声明时不一致，因为 Python 解释器能够用参数名匹配参数值。此外，还有默认参数：调用函数时，缺省参数的值如果没有传入，则被认为是默认值。

例 5.3.5:

```
def plus(str1, str2 = 'science'):  
    total = str1 + str2  
    return total;  
plus('statistical ')
```

'statistical science'

```
plus('mathematical ')
```

'mathmatical science'

```
plus(str1 = 'statistical ')
```

'statistical science'

有时我们可能需要一个函数能处理比当初声明时更多的参数。这些参数叫做不定长参数，和上述 2 种参数不同，声明时不会命名不定长参数。基本语法如下：

```
def functionname( [[ formal_args, ]] * var_args_tuple ):
    function_suite
    return [expression]
```

加了星号 * 的变量名会存放所有未命名的变量参数。

例 5.3.6:

```
def 输出可变长度(arg1, *vartuple):  
    "打印任何传入的参数"  
    print(arg1)  
    print(vartuple)  
    return;
```

可以比较一下如何两个输出结果的差异:

```
输出可变长度(1);
```

```
输出可变长度(1, 2, 3);
```

`return [[ 表达式]]` 退出函数，该语句选择性地调用方返回一个表达式。不带参数值的 `return` 语句返回 `None`。

例 5.3.7：列表反转函数。

```
def 反转1(li):  
    for i in range(0, int(len(li)/2)):  
        temp = li[i]  
        li[i] = li[-i-1]  
        li[-i-1] = temp  
    return li
```

```
li = [1, 2, 3, 4, 5]  
反转1(li)
```

[5, 4, 3, 2, 1]

例 5.3.7

```
def 反转2( list ):  
    for i in range(int(len( list )/2)):  
        list [ i ], list [ -i - 1] = list [ -i - 1], list [ i]  
    return list
```

反转2(li)

[5, 4, 3, 2, 1]

例 5.3.7

```
def 反转3(ListInput):  
    RevList=[]  
    for i in range(len(ListInput)):  
        RevList.append(ListInput.pop())  
    return RevList
```

反转3(li)

[5, 4, 3, 2, 1]

以前我们定义的函数都有对应的函数名，没有名字的函数则被成称为匿名函数。在 Python 中，对匿名函数提供有限支持。当在传入函数时，有时我们无需显式地定义函数，也就不担心函数名冲突的问题了。创建匿名函数需要用到 lambda。这里，lambda 的主题是一个表达式，而不是一个代码块。语句中只能有一个表达式，不用写 return，返回值就是该表达式的结果。因此，lambda 函数体比 def 简单很多，但这同时导致了 lambda 表达式中只能封装非常有限的逻辑。另外，lambda 函数不能访问自有参数列表之外或全局命名空间里的参数。lambda 函数的语法如下：

```
lambda [[ arg1 [[ ,arg2 ,..., argn ]]]] : expression
```

例 5.3.8:

```
lambda x: x * x;
```

```
lambda x, y: x + y;
```

问题在于，匿名函数怎么调用呢？这时候我们就需要使用 map 函数了。

```
map(lambda x: x * x, [1, 4, 9])
```

```
<map at 0x105998860>
```

注：map() 语法是 map(function, iterable, ...)。使用 map() 函数将第一个参数 (function) 作用到第二根参数 (iterable) 上。其中，第一个参数 function 是一个函数，第二个参数 iterable 是一个或多个序列。map() 将 function 函数值返回为一个迭代器，要用 list() 才能正确显示结果。

```
list (map(lambda x, y: x + y, [1,4,7],[2,5,8]) )
```

[3, 9, 15]

匿名函数是一个函数对象，可以把匿名函数赋值给一个变量，再利用变量来调用该函数。

```
square = lambda x: x * x  
square(5)
```


匿名函数

```
del sum
```

```
sum = lambda x, y: x ** 2 + y ** 2
```

```
sum(1,4)
```

17

```
summation = lambda x, y: x + y
```

```
summation(1,2)
```

3

```
circle = lambda x, y: x * x + y * y
```

```
circle (3,4)
```

25

匿名函数

```
def odd(n):  
    return n % 2 == 1  
list ( filter (odd, range(1, 10)))
```

[1, 3, 5, 7, 9]

```
list ( filter (lambda x: x % 2 == 1, range(10)))
```

[1, 3, 5, 7, 9]

匿名函数

```
import math  
  
def sqrit(x):  
    return math.sqrt(x) % 1 == 0  
  
list ( filter ( sqrit , range(1, 10)))
```

[1, 4, 9]

```
list ( filter (lambda x: math.sqrt(x) % 1 == 0, range(1,10)))
```

[1, 4, 9]

上面定义了两个函数的两种不同形式，一种用 `def` 来定义，一种用 `lambda` 来定义。注意，根据 `def` 来定义的函数，`return` 是一个逻辑判断语句。另外，上面的函数都使用了 `filter()` 函数。这个函数用于过滤不符合条件的元素，返回符合条件的元素。语法是 `filter(function, iterable)`。其中，第一个参数 `function` 是判断函数，第二个参数 `iterable` 是可迭代对象。该函数接收序列的每个元素作为参数传递给函数，结果可能为 `True` 或 `False`，最后仅仅返回取值为 `True` 的元素。Python3 中返回到是一个 `filter` 类，需要用 `list` 函数转为列表之后才能正确显示结果。在本章结束之前，我们补充介绍一个 `list` 函数，这个函数在之前被多次使用了，但是我们没有详细结果过。事实上，`list` 函数的功能是将一个序列转换为一个列表。类似的函数还有许多，这些函数可以将数据的类型进行转换，并返回一个新的对象。